# Evidencia del proceso de análisis y diseño

UdeAWorldCup – Simulación del Mundial FIFA 2026

Camilo Andres Villanueva Gonzalez
Jose Manuel Posada Londoño

17 de abril de 2026

## 1. Análisis del problema

Para desarrollar el programa en c++ se hará uso de la programacion orientada a objetos que permitan simular el campeonato Mundial de la FIFA 2026. El sistema manejara información de equipos, jugadores, grupos, partidos y calendario, además de actualizar estadísticas históricas y mostrar resultados del torneo, cumpliendo con las restricciones del documento, como el uso de memoria dinamica, programacion modular y la medicion del consumo de recursos.

Despues de analizar el problema, notamos que no se trata solo de almacenar datos, sino de representar correctamente la logica del torneo, ya que cada equipo posee una informacion propia, como el pais, el ranking FIFA, la confederacion y las estadisticas historicas. Teniendo en cuenta tambien que cada equipo tiene una plantilla de jugadores, donde cada jugar necesita guardar sus estadisticas individuales. Además, el sistema debe ser capaz de organizar grupos, simular partidos, generar clasificaciones y continuar con las fases eliminatorias hasta llegar a la final.

Otro aspecto importante del análisis fue darnos cuenta de que la solución no podía hacerse de forma desordenada o con una sola clase grande, porque eso dificultaría la implementación. Por eso decidímos dividir el problema en varias clases que representaran de forma más natural las entidades del torneo, asi, el programa queda más claro, más modular y más coherente con el paradigma orientado a objetos que se exige en la actividad.

## 2. Propuesta de solución

La solución que se propone se basa en construir un sistema modular en la consola, donde `Torneo` sea la clase principal, esta clase se encarga de coordinar el flujo general del programa y de administrar los equipos, los grupos, los partidos y las diferentes etapas del campeonato.

La clase `Jugador` se encarga de representar a cada futbolista. En ella se guardan atributos como nombre, apellido, número de camiseta y estadísticas acumuladas, como por ejemplo goles, minutos jugados, partidos, tarjetas y faltas. Por lo tanto esta clase permite manejar la información individual de cada jugador y actualizarla después de cada partido.

La clase `Equipo` representa una selección nacional. En esta clase se almacena la información general del equipo, como país, confederación, director técnico, ranking FIFA y estadísticas históricas, tambien, contiene una plantilla dinámica de 26 objetos `Jugador` e incorpora datos temporales para la fase de grupos, como puntos, goles a favor y goles en contra.

La clase `Grupo` se en carga de modelar cada grupo del mundial. Su trabajo principal es almacenar los cuatro equipos asignados y permitir su organización y ordenamiento según el puntaje y los criterios de desempate.

La clase `Partido` tiene como rol representar un encuentro entre dos selecciones. Esta clase guarda datos como hora, fecha, sede, equipos involucrado, árbitros, resultado y estadísticas del partido concentrando la logica principal de simulacion.

La clase `Fecha` se utiliza para manejar de forma separada las operaciones sobre fechas, especialmente para la generación del calendario de la fase de grupos, donde existen restricciones de días entre partidos.

Finalmente, la clase `MedidorRecursos`, cuya función es llevar un control aproximado del número de iteraciones realizadas y de la memoria consumida, siendo asi uno de los requisitos mas importantes a tratar para mejorar la eficiencia del programa.

## 3. Diseño de clases y relaciones

Para el diseño de la solución tratamos de que cada clase tuviera una accion clara, la relación más importante es que un `Equipo` contiene varios `Jugador`, porque la plantilla pertenece directamente al equipo, tambien, un `Grupo` agrupa cuatro equipos, Un `Partido` relaciona dos equipos y registra lo que ocurre durante el encuentro, y encima de estas clases, la clase `Torneo` administra la lógica global de la copa.

Las relaciones principales del diseño son las siguientes:

- `Torneo` administra todos los `Equipo`.
- `Torneo` administra todos los `Grupo`.
- `Torneo` administra los `Partido` de cada etapa.
- `Equipo` contiene una plantilla de `Jugador`.
- `Grupo` agrupa cuatro objetos `Equipo`.
- `Partido` enfrenta dos objetos `Equipo`.
- `Torneo` usa `MedidorRecursos` para reportar eficiencia.

Con este diseño buscamos que la solución no fuera trivial, sino que realmente reflejara el problema planteado, tambien, esta organización facilita trabajar en múltiples archivos `.h` y `.cpp`.

## 4. Algoritmos principales

Uno de los primeros algoritmos necesarios es la carga de los datos históricos de los equipos, para esto se debe leer el archivo csv dado y construir a partir de él los objetos `Equipo`, despues, para cada selección se genera artificialmente una plantilla de 26 jugadores con nombres, apellidos, números consecutivos de camiseta y estadísticas iniciales en cero y los goles históricos del equipo se reparten de manera uniforme entre los jugadores.

Otro algoritmo importante es la conformación de los bombos y el sorteo de grupos, primero, el país anfitrión se ubica directamente en el bombo 1 y luego el resto de selecciones se organiza según el

ranking fifa, y después se realiza un sorteo aleatorio respetando la restricción de confederaciones, que nos dice que no puede haber dos equipos de la misma confederación en un grupo, excepto UEFA, que puede tener máximo dos.

Otro algoritmo importante es el de simulación de partidos, en este punto, se deben escoger 11 jugadores por equipo, estimar la cantidad de goles esperados y simular estadísticas como posesión, faltas y tarjetas, despues de cada partido se actualizaran tanto las estadísticas históricas del equipo como las estadísticas individuales de los jugadores.

Y el ultimo algoritmo es la clasificación de la fase de grupos y el avance por el fixture, Para esto se ordenan los equipos de cada grupo según los puntos que tengan, la diferencia de goles y los goles a favor, y a partir de esto se seleccionaran los clasificados a la siguiente ronda, después se construiran las fases eliminatorias hasta definir el campeón, el subcampeón y el resto de posiciones finales.

## 5. Consideraciones de implementación

Sobre la implentación, decidimos usar punteros y arreglos dinamicos, por lo tanto lo mas importante es prestar atencion y llevar el conteo del manejo de memoria, especialmente en las clases que contienen atributos dinamicos, como Jugador, Equipo y Partido. Por esto mismo, consideramos generar la implementacion correcta del constructor de copia, el destructor y el operador de asignación.

Ademas de esto, se busca que la solución fuera eficiente desde el diseño ya que al tener varias cantidades del problema conocidas, se nos facilita justificar la elección de estructuras de datos simples y adecuadas al problema, por ejemplo 48 equipos, 12 grupos, 26 jugadores por selección y 11 convocados por partido. Por lo tanto, resulta razonable usar arreglos propios de tamaños controlados.

Tambien se planea medir el consumo de recursos al finalizar cada funcion importante, mostrando la cantidad de iteraciones realizadas y una aproximación de la memoria consumida por las estructuras existentes en el sistema.

## 6. Dificultades previstas

Notamos que una de las mayores dificultades del proyecto está en coordinar correctamente la interacción entre todas las partes del sistema. No basta con que cada clase funcione por separado; también es necesario que la información permanezca consistente entre la carga inicial, la simulación de partidos, la actualización de estadísticas y el avance del torneo.

Otra dificultad importante es el manejo de memoria dinámica, ya que el proyecto depende de punteros, arreglos dinámicos y relaciones entre objetos. Además, si la generación de fechas no se trata de la forma correcta para la fase de grupos puede ser un punto delicado, ya que esta debe respetar restricciones temporales del enunciado.

Por esta razón nuestra idea es desarrollar la solución de manera incremental, es decir, primero consolidar el modelo de clases, luego implementar la carga de datos y la formación de grupos, y finalmente completar la simulación de partidos, las rondas eliminatorias y las estadísticas finales. De esta manera, el sistema puede ir validándose poco a poco. (Divide y Venceras)

## 7. Diagrama de clases

Aca se incluye el diagrama de clases de la solución propuesta, en él se representan las clases principales del sistema: `Jugador`, `Equipo`, `Grupo`, `Fecha`, `Partido`, `MedidorRecursos` y `Torneo`, junto con sus relaciones.

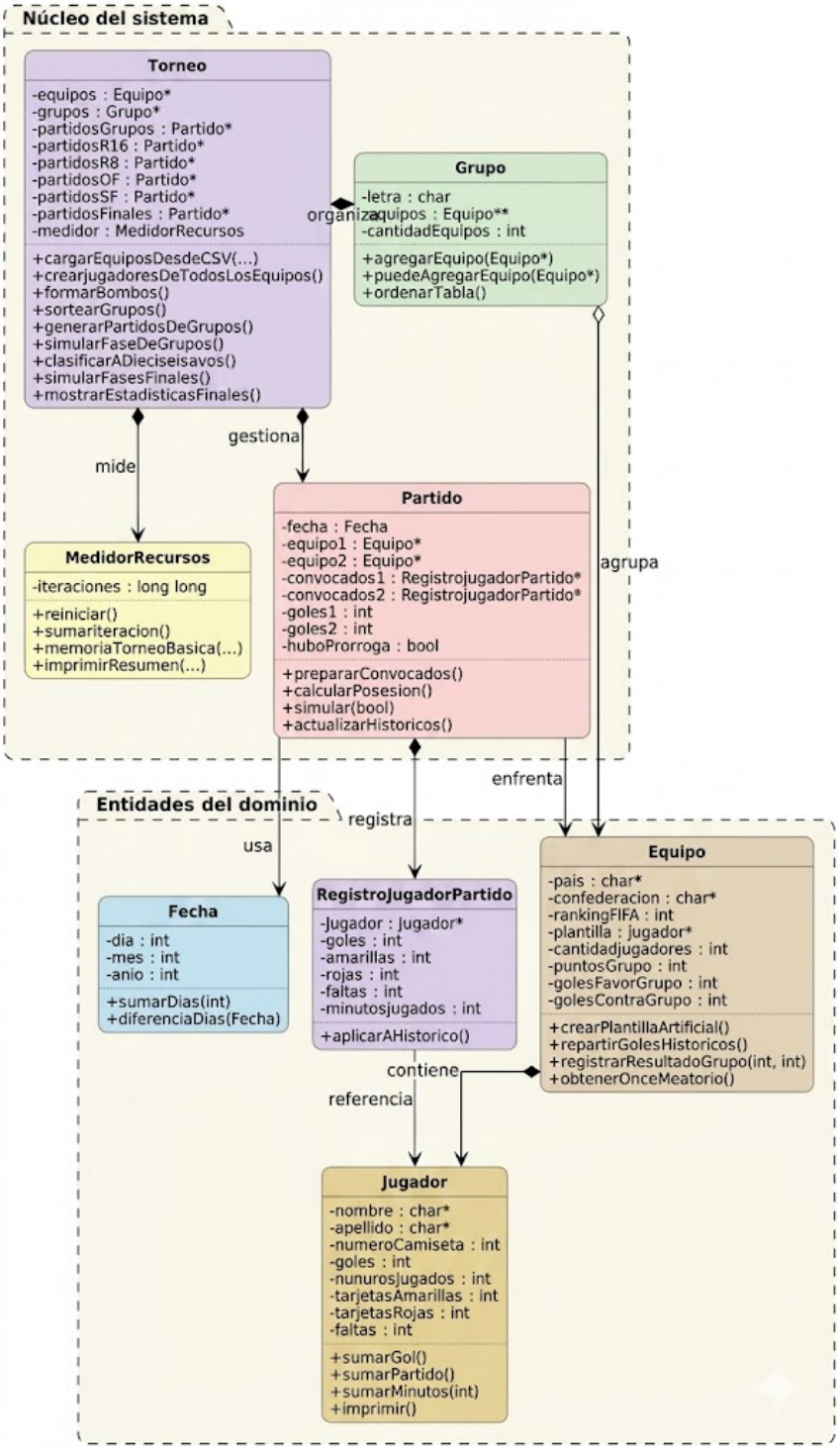


Figura 1: Diagrama de clase del Mundial FIFA 2026.